



SoftDomiFood

Resultado del análisis de contexto y prototipo final generado por Agent IRIS

Generado: 15 de diciembre de 2025, 19:27

Versión: 1.0

Resumen del Proceso

El proceso de análisis ha sido completado exitosamente. Aquí tienes un resumen de los resultados principales:

Tabla de Contenido

1. Análisis y síntesis de contexto

1.. Análisis y síntesis de contexto

- 1.1. Resumen ejecutivo
- 1.2. Propósito del proyecto
- 1.3. Objetivo principal del proyecto
- 1.4. Descripción del alcance del proyecto
- 1.5. Flujo principal del proceso

- 1.5.1 **Interacción del Cliente y Creación de Pedido:**
- 1.5.2 **Procesamiento Inicial del Pedido en el Backend:**
- 1.5.3 **Comunicación Asíncrona y Procesamiento por el Worker:**
- 1.5.4 **Seguimiento y Gestión del Pedido:**
- 1.6. **Análisis de requisitos y contexto de negocio**
 - 1 Requisitos funcionales
 - 1 Requisitos no funcionales
 - 1 Dentro del alcance
 - 1 Fuera del alcance
- 1.7. **Criterios de aceptación generales**
- 1.8. **Criterios de no aceptación generales**
- 1.9. **Supuestos**
- 1.10. **Restricciones**
- 1.11. **Listado de módulos**
- 1.12. **Dependencias identificadas**

2.. Inventario de épicas, features e historias de usuario

- 2.1. **Épicas**
 - 2.1.1. **Épica 1**
 - 2.1.2. **Épica 2**
- 2.2. **Features**
 - 2.2.1. **Feature 1**
 - 2.2.2. **Feature 2**
 - 2.2.3. **Feature 3**
 - 2.2.4. **Feature 4**
 - 2.2.5. **Feature 5**
 - 2.2.6. **Feature 6**

- 2.2.7. Feature 7
- 2.2.8. Feature 8
- 2.2.9. Feature 9
- 2.2.10. Feature 10
- 2.2.11. Feature 11
- 2.2.12. Feature 12
- 2.2.13. Feature 13
- 2.2.14. Feature 14
- 2.2.15. Feature 15
- 2.2.16. Feature 16
- 2.2.17. Feature 17
- 2.2.18. Feature 18
- 2.3. Historias de usuario
 - 2.3.1. Historia de usuario 1
 - 2.3.2. Historia de usuario 2
 - 2.3.3. Historia de usuario 3
 - 2.3.4. Historia de usuario 4
 - 2.3.5. Historia de usuario 5
 - 2.3.6. Historia de usuario 6
 - 2.3.7. Historia de usuario 7
 - 2.3.8. Historia de usuario 8
 - 2.3.9. Historia de usuario 9
 - 2.3.10. Historia de usuario 10
 - 2.3.11. Historia de usuario 11
 - 2.3.12. Historia de usuario 12
 - 2.3.13. Historia de usuario 13
 - 2.3.14. Historia de usuario 14
 - 2.3.15. Historia de usuario 15

- 2.3.16. Historia de usuario 16
- 2.3.17. Historia de usuario 17
- 2.3.18. Historia de usuario 18
- 2.3.19. Historia de usuario 19
- 2.3.20. Historia de usuario 20
- 2.3.21. Historia de usuario 21
- 2.3.22. Historia de usuario 22
- 2.3.23. Historia de usuario 23
- 2.3.24. Historia de usuario 24
- 2.3.25. Historia de usuario 25
- 2.3.26. Historia de usuario 26
- 2.3.27. Historia de usuario 27
- 2.3.28. Historia de usuario 28
- 2.3.29. Historia de usuario 29
- 2.3.30. Historia de usuario 30
- 2.3.31. Historia de usuario 31
- 2.3.32. Historia de usuario 32
- 2.3.33. Historia de usuario 33
- 2.3.34. Historia de usuario 34
- 2.3.35. Historia de usuario 35
- 2.3.36. Historia de usuario 36
- 2.3.37. Historia de usuario 37
- 2.3.38. Historia de usuario 38
- 2.3.39. Historia de usuario 39
- 2.3.40. Historia de usuario 40
- 2.3.41. Historia de usuario 41
- 2.3.42. Historia de usuario 42
- 2.4. Matriz de riesgos

- 2.4.1. Matriz de evaluación de riesgos

3. Información del Proyecto

Análisis y síntesis de contexto

1. Análisis y síntesis de contexto

1.1. Resumen ejecutivo

SoftDomiFood es una plataforma de pedidos de comida a domicilio que permite a los usuarios gestionar pedidos, productos y la logística de entrega o recogida, con el objetivo de establecer un restaurante en línea robusto. Actualmente, el sistema, aunque funcional y basado en una arquitectura de microservicios, presenta deficiencias críticas en seguridad, mantenibilidad, escalabilidad y operación en producción. Los principales problemas identificados incluyen la duplicación de backends con lógica redundante, malas prácticas de seguridad (ej. secretos hardcodeados, JWT inseguros), y una carencia significativa de automatización (CI/CD) y observabilidad. El proyecto se enfoca en corregir estas deficiencias estructurales para garantizar la estabilidad, seguridad y evolución futura de la plataforma.

1.2. Propósito del proyecto

El propósito fundamental del proyecto SoftDomiFood es proporcionar una plataforma digital completa para la gestión de pedidos de un restaurante en línea. Busca resolver la necesidad de los usuarios de realizar pedidos de comida de manera conveniente, ya sea para recogida en el local o para entrega a domicilio, optimizando la experiencia del cliente y la operación interna del restaurante mediante un sistema seguro, mantenible y escalable.

1.3. Objetivo principal del proyecto

El objetivo principal del proyecto es establecer una plataforma de pedidos en línea robusta y segura para el restaurante, que permita a los clientes realizar y gestionar sus pedidos (para recogida o domicilio) de manera eficiente, y a los administradores gestionar productos, pedidos, cupones y reseñas, asegurando la mantenibilidad, escalabilidad y seguridad del sistema mediante la resolución de la deuda técnica crítica y la implementación de mejores prácticas de desarrollo y operación.

1.4. Descripción del alcance del proyecto

El alcance del proyecto SoftDomiFood abarca el desarrollo y mejora de una plataforma de pedidos en línea para un restaurante, incluyendo:

- **Interfaz de Usuario Cliente:** Un frontend web que permite a los clientes navegar el menú, agregar productos al carrito, gestionar favoritos (añadir y eliminar), crear pedidos (directos o programados), aplicar cupones, ver el estado de sus pedidos y dejar reseñas.

- **Interfaz de Administración:** Un panel web para administradores que facilita la gestión de productos (creación, edición, eliminación), el seguimiento y actualización del estado de los pedidos, la creación y gestión de cupones, la visualización y eliminación de reseñas, y la consulta del historial de clientes.
- **Backend / API:** Un conjunto de servicios (API principal en FastAPI) para gestionar la autenticación, productos, pedidos, direcciones, cupones y reseñas, incluyendo la consolidación de la lógica duplicada actualmente existente.
- **Base de Datos:** Implementación y gestión de una base de datos relacional (PostgreSQL) para almacenar toda la información del negocio, incluyendo scripts para inicialización y recuperación.
- **Procesamiento Asíncrono:** Un worker (Node.js + TypeScript) para el procesamiento asíncrono de eventos de pedidos, utilizando RabbitMQ como broker de mensajería.
- **Infraestructura:** Uso de Podman para la orquestación de servicios y un entorno de desarrollo consistente.
- **Seguridad:** Implementación de prácticas de seguridad robustas, incluyendo la externalización y rotación de secretos, y el fortalecimiento de la gestión de JWT.
- **Calidad y Automatización:** Establecimiento de pipelines de CI/CD (GitHub Actions) para automatización de pruebas (unitarias, de integración y seguridad), linters automáticos y pre-commit hooks para asegurar la calidad del código y la detección temprana de errores.
- **Observabilidad:** Implementación de logging estructurado, métricas, trazabilidad distribuida y alertas para monitorear el estado y rendimiento del sistema en producción.

1.5. Flujo principal del proceso

1.5.1 Interacción del Cliente y Creación de Pedido:

- El cliente navega por el menú de productos disponibles a través del frontend.

- Selecciona los productos deseados y los agrega a su carrito de compras.
- Opcionalmente, puede marcar productos como favoritos o eliminarlos de su lista de favoritos para futuras compras.
- Al proceder con la compra, el cliente selecciona una dirección de entrega o indica si el pedido es para recogida.
- El cliente decide si el pedido es "directo" (para ser preparado inmediatamente) o "para más tarde" (programado, con un límite de 48 horas).
- Selecciona el medio de pago (ej. efectivo).
- Confirma el pedido, enviando la solicitud al backend.

1.5.2 Procesamiento Inicial del Pedido en el Backend:

- El backend (API principal, FastAPI) recibe la solicitud del pedido.
- Valida la autenticación del cliente mediante el token JWT.
- Realiza las validaciones de negocio pertinentes (ej. horario del restaurante, disponibilidad de productos, validez del cupón si aplica).
- Guarda los detalles del pedido en la base de datos PostgreSQL con un estado inicial (ej. "pendiente").

1.5.3 Comunicación Asíncrona y Procesamiento por el Worker:

- Una vez guardado el pedido, el backend publica un evento de "Pedido Creado" en el broker de mensajería RabbitMQ.
- El módulo Worker consume el mensaje de RabbitMQ.
- El Worker procesa el evento y realiza acciones asíncronas, como actualizar el estado detallado del pedido en la base de datos.

1.5.4 Seguimiento y Gestión del Pedido:

- El cliente puede consultar el estado de su pedido a través del frontend.
- El frontend cliente realiza un polling cada 10 segundos a un servicio tipo "Get Orders" para obtener actualizaciones del estado del pedido (pendiente, en preparación, listo, entregado, programado).
- Desde el panel administrativo, el administrador puede visualizar todos los pedidos.
- El administrador puede cambiar el estado de los pedidos (ej. "en preparación", "listo", "entregado"). Para pedidos programados, los botones de gestión de estado solo aparecen una vez que se cumple la fecha/hora programada, permitiendo solo la cancelación antes de ese momento.

1.6. Análisis de requisitos y contexto de negocio

Requisitos funcionales

- **Gestión de Cuentas:**
 - El sistema debe permitir el registro y inicio de sesión de clientes.
 - El sistema debe permitir el inicio de sesión de administradores con credenciales predefinidas.
- **Gestión de Productos (Administrador):**
 - El administrador debe poder crear, editar y eliminar productos.
 - El sistema debe permitir la carga inicial de productos mediante un proceso de "semilla".
- **Navegación y Selección de Productos (Cliente):**
 - El cliente debe poder visualizar el menú de productos disponibles.
 - El cliente debe poder agregar productos al carrito de compras.

- El cliente debe poder marcar productos como favoritos, visualizar su lista de favoritos y eliminar productos de su lista de favoritos.
- El cliente debe poder visualizar las reseñas de los productos en el menú.
- **Gestión de Pedidos (Cliente):**
 - El cliente debe poder crear un pedido seleccionando una dirección de entrega.
 - El cliente debe poder elegir si el pedido es para entrega "directa" o "programada" (para más tarde).
 - El cliente debe poder seleccionar el medio de pago (ej. efectivo).
 - El cliente debe poder aplicar cupones de descuento al carrito de compras.
 - El cliente debe poder consultar el estado de sus pedidos (pendiente, en preparación, listo, entregado, programado).
 - El cliente debe poder calificar un producto una vez que el pedido ha sido entregado.
- **Gestión de Pedidos (Administrador):**
 - El administrador debe poder visualizar todos los pedidos.
 - El administrador debe poder cambiar el estado de los pedidos (pendiente, en preparación, listo, entregado, programado).
 - El administrador debe poder cancelar pedidos programados antes de su fecha/hora de ejecución.
- **Gestión de Cupones (Administrador):**
 - El administrador debe poder crear y actualizar cupones de descuento.
- **Gestión de Reseñas (Administrador):**
 - El administrador debe poder listar y eliminar reseñas de productos.
- **Gestión de Clientes (Administrador):**
 - El administrador debe poder visualizar el historial de pedidos de los clientes.

Requisitos no funcionales

- **Seguridad:**

- El sistema debe proteger los secretos (contraseñas, JWT) y no almacenarlos hardcoded en el código o podman-compose.
- El sistema debe utilizar tokens JWT seguros y con secretos robustos.
- El sistema debe proteger los tokens de sesión de vulnerabilidades XSS y CSRF (no almacenar tokens en localStorage).
- El sistema debe implementar rotación de credenciales.
- **Mantenibilidad y Calidad de Código:**
- El sistema debe eliminar la duplicación de lógica entre backends (consolidar `api/` y `backend/`).
- Las funciones deben adherirse al principio de responsabilidad única.
- El código no debe usar `print()` o `console.log()` para logging en producción.
- El código TypeScript no debe usar `any` de forma indiscriminada.
- El sistema debe evitar SQL embebido en strings.
- El sistema debe implementar un manejo de errores robusto y específico, no genérico.
- **Escalabilidad y Rendimiento:**
- El sistema debe permitir el procesamiento asíncrono de pedidos para desacoplar la creación del pedido de su procesamiento.
- El sistema debe implementar una estrategia de caché para mejorar el rendimiento.
- El sistema debe ser desplegable y escalable mediante Podman.
- **Confiabilidad y Operación:**
- El sistema debe tener esquemas de datos consistentes entre sus componentes (Prisma sincronizado).
- El sistema debe implementar pipelines de Integración Continua (CI) y Despliegue Continuo (CD) para automatizar validación y despliegues.
- El sistema debe contar con observabilidad completa (logs estructurados, métricas, trazabilidad distribuida, alertas).
- El sistema debe implementar health checks para sus servicios.
- El sistema debe manejar mensajes no procesados en RabbitMQ (Dead Letter Queue - DLQ).

- **Pruebas:**
- El sistema debe tener cobertura de pruebas unitarias, de integración y End-to-End (E2E), incluyendo el flujo crítico de pedidos y el worker.
- El sistema debe integrar herramientas de análisis estático de seguridad (SAST) de forma automatizada.
- **Experiencia de Desarrollo:**
- El sistema debe integrar linters automáticos y pre-commit hooks para asegurar la calidad del código antes de la subida a repositorio.

Dentro del alcance

- Desarrollo de dos interfaces web: cliente y administración.
- Implementación de un backend principal con FastAPI (Python) para gestionar la lógica de negocio.
- Uso de PostgreSQL como base de datos relacional.
- Utilización de RabbitMQ para mensajería asíncrona y un worker basado en Node.js/TypeScript.
- Containerización de todos los servicios mediante Podman.
- Gestión de productos, pedidos, cupones, reseñas y cuentas de usuario.
- Procesos de carga de datos iniciales (semillas).
- Implementación de seguridad básica y avanzada (gestión de secretos, JWT).
- Estrategias de testing (unitario, integración, E2E, SAST).
- Configuración de CI/CD con GitHub Actions.
- Implementación de observabilidad (logs, métricas, tracing, alertas).
- Consolidación del backend eliminando el duplicado de Express.

Fuera del alcance

- Desarrollo de aplicaciones móviles nativas para cliente o administrador.
- Integración con múltiples pasarelas de pago más allá de lo básico (ej. efectivo).
- Implementación de funcionalidades avanzadas de chat en tiempo real para soporte.
- Optimización de la experiencia de usuario en casos especiales no críticos (ej. cierre automático de modal "agregar producto", ajuste de CSS en reseñas).
- Refactorización completa de "muchos archivos no probados/por depurar" que no son parte del core.
- Validación de la funcionalidad "CUA" mencionada si no se detalla su propósito y requisitos.

1.7. Criterios de aceptación generales

- La plataforma debe permitir a los clientes registrarse, iniciar sesión y realizar pedidos exitosamente, tanto directos como programados.
- Los pedidos deben ser procesados y su estado actualizado correctamente a través del flujo Producer-Consumer.
- El administrador debe poder gestionar productos, cupones, pedidos y reseñas de forma completa y consistente.
- Los secretos del sistema (contraseñas, JWT) deben estar externalizados y no ser accesibles en el repositorio de código.
- Todos los componentes del backend duplicado deben ser eliminados y su lógica consolidada en el backend principal (FastAPI).
- Debe existir un pipeline de CI/CD funcional que ejecute tests (unitarios, de integración, seguridad) y linting de forma automática.
- El sistema debe generar logs estructurados y métricas de operación.
- El flujo de creación y seguimiento de pedidos debe estar cubierto por pruebas automatizadas.

- El tiempo máximo para programar un pedido no debe exceder las 48 horas.
- Los botones de gestión de estado para pedidos programados solo deben activarse en la fecha/hora correspondiente.

1.8. Criterios de no aceptación generales

- La presencia de secretos hardcodeados en el repositorio de código o archivos de configuración accesibles públicamente.
- La existencia del backend duplicado (`backend/` Express) con lógica de negocio activa.
- Fallos en el proceso de registro, inicio de sesión o creación de pedidos de clientes.
- Ausencia de pruebas automatizadas para el flujo crítico de creación y seguimiento de pedidos.
- Incumplimiento del límite de 48 horas para la programación de pedidos.
- Errores de seguridad críticos como vulnerabilidades XSS o CSRF explotables debido a la gestión de tokens.
- Inconsistencias en el esquema de la base de datos entre los servicios (backend y worker).
- Falta de observabilidad (logs, métricas) que impida el monitoreo efectivo del sistema en producción.

1.9. Supuestos

- Se asume que la arquitectura de microservicios existente, con un patrón Producer-Consumer, es la base adecuada para la evolución del sistema.
- Se asume que FastAPI (Python) será el framework elegido para la consolidación del backend, archivando el backend de Express.
- Se asume que GitHub Actions será la plataforma para implementar CI/CD.

- Se asume que el horario de cierre del restaurante es a las 23:00 horas.
- Se asume que la base de datos PostgreSQL actual es la adecuada y no requiere un cambio fundamental de tecnología.
- Se asume que el equipo de desarrollo tiene la capacidad y el conocimiento para implementar las mejoras de seguridad, testing y observabilidad recomendadas.
- Se asume que el sistema operará en un entorno web, sin requisitos de aplicaciones móviles nativas.

1.10. Restricciones

- **Tecnológicas:**
 - La plataforma debe utilizar Podman para el despliegue y orquestación de servicios.
 - El backend principal debe ser desarrollado en Python con FastAPI.
 - El worker debe continuar utilizando Node.js y TypeScript.
 - La base de datos principal debe ser PostgreSQL.
 - RabbitMQ es el broker de mensajería establecido.
 - El frontend cliente y admin deben ser desarrollados con React 18 + Vite y Tailwind CSS.
- **Tiempo:**
 - La programación de pedidos para "más tarde" no puede superar las 48 horas desde el momento de la creación.
 - El restaurante tiene un horario de operación definido, abriendo a las 10:00 a.m. y cerrando a las 23:00 horas, lo que restringe la disponibilidad para pedidos.
- **Seguridad:**
 - No se permite el almacenamiento de secretos directamente en el código fuente o en archivos de configuración versionados.
 - Los tokens de autenticación (JWT) no deben ser almacenados en `localStorage` del cliente debido a vulnerabilidades XSS.

- **Negocio:**
- El cambio de estado de los pedidos no es en tiempo real; se basa en un mecanismo de polling cada 10 segundos desde el frontend.
- Los cupones pueden ofrecer descuento por porcentaje o monto fijo.
- Los botones de gestión de estado para pedidos programados no deben aparecer hasta que la fecha/hora programada se cumpla.

1.11. Listado de módulos

- **api/ (Backend Principal):** Responsable de la autenticación de usuarios, gestión de productos, procesamiento de pedidos, gestión de direcciones, cupones y reseñas. Desarrollado en FastAPI (Python). Será el backend único tras la consolidación.
- **worker/ (Procesador Asíncrono):** Consume mensajes de RabbitMQ para el procesamiento asíncrono de eventos de pedidos y la actualización de su estado. Desarrollado en Node.js + TypeScript.
- **frontend/ (Interfaz Cliente):** Proporciona la interfaz web para los usuarios finales, permitiendo la navegación de menú, carrito, creación de pedidos, gestión de favoritos (añadir y eliminar), aplicación de cupones y visualización de reseñas y estado de pedidos. Desarrollado en React 18 + Vite.
- **admin-frontend/ (Panel Administrativo):** Ofrece la interfaz web para administradores, facilitando la gestión de productos, pedidos, estados, cupones, reseñas y la visualización del historial de clientes. Desarrollado en React 18 + Vite.
- **database/ (Base de Datos):** Contiene la base de datos relacional PostgreSQL 15, incluyendo scripts para backups SQL, inicialización y recuperación de datos.
- **qa_automated/ (Pruebas Automatizadas):** Módulo dedicado a las pruebas automatizadas del sistema, incluyendo unitarias, de integración, E2E y análisis de seguridad (SAST).
- **scripts/ (Scripts de Operación):** Contiene scripts de PowerShell para tareas operativas y de desarrollo.

- **podman-compose.yml:** Archivo de configuración para la orquestación y despliegue de los servicios mediante Podman Compose.

1.12. Dependencias identificadas

- **Consolidación de Backend:** El desarrollo de nuevas funcionalidades en el backend principal depende de la eliminación exitosa y la migración de la lógica del backend duplicado (**backend/** Express).
- **Gestión de Secretos:** La implementación de seguridad robusta depende de la disponibilidad de un sistema para la gestión y rotación de secretos (ej. un Secrets Manager o variables de entorno robustas).
- **CI/CD:** La automatización de la calidad y los despliegues depende de la configuración de GitHub Actions y la definición de los flujos de trabajo de CI/CD.
- **Testing:** La validación de la estabilidad del sistema depende de la implementación completa de pruebas unitarias, de integración y E2E, especialmente para el flujo de pedidos.
- **Sincronización de Esquemas:** La correcta operación del worker y el backend depende de que los esquemas de datos de Prisma estén consistentemente sincronizados entre ambos.
- **Recursos Humanos:** La capacidad de implementar las mejoras y resolver la deuda técnica depende de la disponibilidad de personal con experiencia en FastAPI, Node.js/TypeScript, seguridad, DevOps y testing.
- **Definición de Horario:** La implementación precisa de las restricciones de horario del restaurante se basa en la hora de cierre definida a las 23:00 horas.
- **Resolución de Desfase Horario:** La correcta programación de pedidos depende de la resolución del posible problema de zona horaria que causa un desfase de ~3 horas.

¿Es correcto mi entendimiento?, si es así, dar click al botón **Siguiente fase** .

2. Inventario de épicas, features e historias de usuario

2.1. Épicas

2.1.1. Épica 1

Título: Plataforma de Pedidos en Línea Robusta y Segura

ID épica: EP-001

Descripción: Establecer una plataforma digital completa para la gestión de pedidos de un restaurante en línea, optimizando la experiencia del cliente y la operación interna mediante un sistema seguro, mantenible y escalable.

Narrativa / Descripción (El Qué y el Para Quién): Para clientes y administradores de restaurantes, quienes quieren/necesitan **gestionar pedidos de comida de manera eficiente y segura**, la Épica **Plataforma de Pedidos en Línea Robusta y Segura** es un **sistema integral** que proporciona **una experiencia de usuario fluida y una operación interna optimizada con alta seguridad**.

Features:

- FT-001: Gestión de Cuentas de Usuario
- FT-002: Interfaz de Usuario Cliente
- FT-003: Gestión de Pedidos (Cliente)
- FT-004: Interfaz de Administración
- FT-005: Gestión de Productos (Administrador)

- FT-006: Gestión de Pedidos (Administrador)
- FT-007: Gestión de Cupones (Administrador)
- FT-008: Gestión de Reseñas (Administrador)
- FT-009: Gestión de Clientes (Administrador)
- FT-010: Backend / API Principal
- FT-011: Base de Datos
- FT-012: Procesamiento Asíncrono de Pedidos

2.1.2. Épica 2

Título: Optimización de la Operación y Calidad del Sistema

ID épica: EP-002

Descripción: Corregir deficiencias estructurales en seguridad, mantenibilidad, escalabilidad y operación en producción, implementando mejores prácticas de desarrollo y observabilidad.

Narrativa / descripción (el qué y el para quién): Para el equipo de desarrollo y operaciones, quienes quieren/necesitan **garantizar la estabilidad, seguridad y evolución futura de la plataforma**, la Épica **Optimización de la Operación y Calidad del Sistema** es un **conjunto de mejoras técnicas y procesos** que proporciona **un sistema mantenible, escalable, seguro y con alta observabilidad**.

Features:

- FT-013: Seguridad del Sistema
- FT-014: Mantenibilidad y Calidad de Código
- FT-015: Escalabilidad y Rendimiento
- FT-016: Confiabilidad y Operación
- FT-017: Pruebas Automatizadas
- FT-018: Experiencia de Desarrollo

2.2. Features

2.2.1. Feature 1

Identificador único (ID): FT-001

Título / nombre: Gestión de Cuentas de Usuario

Descripción: Permite a los clientes registrarse e iniciar sesión, y a los administradores acceder al sistema con credenciales predefinidas. Incluye requisitos de complejidad para contraseñas de clientes.

Épica: EP-001 - Plataforma de Pedidos en Línea Robusta y Segura

Prioridad: Alta

Historias de usuario (HUs):

- US-001: Registrarse como cliente
- US-002: Iniciar sesión como cliente/administrador

2.2.2. Feature 2

Identificador único (ID): FT-002

Título / Nombre: Interfaz de Usuario Cliente

Descripción: Proporciona la interfaz web para que los clientes naveguen el menú, gestionen el carrito, favoritos y visualicen reseñas.

Épica: EP-001 - Plataforma de Pedidos en Línea Robusta y Segura

Prioridad: Alta

Historias de Usuario (HUs):

- US-003: Visualizar el menú de productos
- US-004: Agregar productos al carrito
- US-005: Gestionar productos favoritos

2.2.3. Feature 3

Identificador único (ID): FT-003

Título / Nombre: Gestión de Pedidos (Cliente)

Descripción: Permite a los clientes crear pedidos (directos o programados con un rango de tiempo específico), seleccionar dirección y pago (inicialmente solo efectivo), aplicar cupones, consultar estado y calificar productos.

Épica: EP-001 - Plataforma de Pedidos en Línea Robusta y Segura

Prioridad: Alta

Historias de Usuario (HUs):

- US-006: Crear un pedido con dirección de entrega
- US-007: Elegir tipo de pedido (directo o programado)
- US-008: Seleccionar medio de pago
- US-009: Aplicar cupones de descuento
- US-010: Consultar el estado de mis pedidos
- US-011: Calificar un producto

2.2.4. Feature 4

Identificador único (ID): FT-004

Título / Nombre: Interfaz de Administración

Descripción: Proporciona el panel web para que los administradores gestionen productos, pedidos, cupones, reseñas y clientes.

Épica: EP-001 - Plataforma de Pedidos en Línea Robusta y Segura

Prioridad: Media

Historias de Usuario (HUs):

- (Las HUs para esta feature se detallan en las features de gestión específicas para el administrador)

2.2.5. Feature 5

Identificador único (ID): FT-005

Título / Nombre: Gestión de Productos (Administrador)

Descripción: Permite a los administradores crear, editar y eliminar productos, así como realizar la carga inicial de productos.

Épica: EP-001 - Plataforma de Pedidos en Línea Robusta y Segura

Prioridad: Alta

Historias de Usuario (HUs):

- US-012: Crear, editar y eliminar productos
- US-013: Cargar productos iniciales (semilla)

2.2.6. Feature 6

Identificador único (ID): FT-006

Título / Nombre: Gestión de Pedidos (Administrador)

Descripción: Permite a los administradores visualizar todos los pedidos, cambiar su estado y cancelar pedidos programados.

Épica: EP-001 - Plataforma de Pedidos en Línea Robusta y Segura

Prioridad: Alta

Historias de Usuario (HUs):

- US-014: Visualizar todos los pedidos
- US-015: Cambiar el estado de los pedidos
- US-016: Cancelar pedidos programados

2.2.7. Feature 7

Identificador único (ID): FT-007

Título / Nombre: Gestión de Cupones (Administrador)

Descripción: Permite a los administradores crear y actualizar cupones de descuento.

Épica: EP-001 - Plataforma de Pedidos en Línea Robusta y Segura

Prioridad: Media

Historias de Usuario (HUs):

- US-017: Crear y actualizar cupones de descuento

2.2.8. Feature 8

Identificador único (ID): FT-008

Título / Nombre: Gestión de Reseñas (Administrador)

Descripción: Permite a los administradores listar y eliminar reseñas de productos.

Épica: EP-001 - Plataforma de Pedidos en Línea Robusta y Segura

Prioridad: Media

Historias de Usuario (HUs):

- US-018: Listar y eliminar reseñas de productos

2.2.9. Feature 9

Identificador único (ID): FT-009

Título / Nombre: Gestión de Clientes (Administrador)

Descripción: Permite a los administradores visualizar el historial de pedidos de los clientes.

Épica: EP-001 - Plataforma de Pedidos en Línea Robusta y Segura

Prioridad: Baja

Historias de Usuario (HUs):

- US-019: Visualizar historial de pedidos de clientes

2.2.10. Feature 10

Identificador único (ID): FT-010

Título / Nombre: Backend / API Principal

Descripción: Consolidación de la lógica de negocio en un único backend FastAPI para gestionar autenticación, productos, pedidos, direcciones, cupones y reseñas.

Épica: EP-001 - Plataforma de Pedidos en Línea Robusta y Segura

Prioridad: Alta

Historias de Usuario (HUs):

- US-020: Consolidar lógica de backend duplicada

2.2.11. Feature 11

Identificador único (ID): FT-011

Título / Nombre: Base de Datos

Descripción: Implementación y gestión de una base de datos relacional PostgreSQL para almacenar toda la información del negocio, incluyendo scripts de inicialización y recuperación.

Épica: EP-001 - Plataforma de Pedidos en Línea Robusta y Segura

Prioridad: Alta

Historias de Usuario (HUs):

- US-021: Mantener esquemas de datos consistentes

2.2.12. Feature 12

Identificador único (ID): FT-012

Título / Nombre: Procesamiento Asíncrono de Pedidos

Descripción: Implementación de un worker (Node.js + TypeScript) y RabbitMQ para el procesamiento asíncrono de eventos de pedidos, desacoplando la creación del pedido de su procesamiento.

Épica: EP-001 - Plataforma de Pedidos en Línea Robusta y Segura

Prioridad: Alta

Historias de Usuario (HUs):

- US-022: Procesar eventos de pedidos asincrónicamente

2.2.13. Feature 13

Identificador único (ID): FT-013

Título / Nombre: Seguridad del Sistema

Descripción: Implementación de prácticas de seguridad robustas, incluyendo externalización y rotación de secretos, fortalecimiento de JWT con duraciones específicas (Access Token: 15 min, Refresh Token: 7 días) y protección contra XSS/CSRF.

Épica: EP-002 - Optimización de la Operación y Calidad del Sistema

Prioridad: Alta

Historias de Usuario (HUs):

- US-023: Externalizar y rotar secretos
- US-024: Utilizar tokens JWT seguros
- US-025: Proteger tokens de sesión contra XSS/CSRF
- US-026: Implementar rotación de credenciales

2.2.14. Feature 14

Identificador único (ID): FT-014

Título / Nombre: Mantenibilidad y Calidad de Código

Descripción: Eliminación de duplicación de lógica, adherencia a principios de diseño, logging estructurado, uso adecuado de tipos y manejo de errores robusto.

Épica: EP-002 - Optimización de la Operación y Calidad del Sistema

Prioridad: Alta

Historias de Usuario (HUs):

- US-027: Eliminar duplicación de lógica entre backends
- US-028: Adherirse al principio de responsabilidad única
- US-029: Usar logging estructurado en producción
- US-030: Evitar el uso indiscriminado de `any` en TypeScript
- US-031: Evitar SQL embebido en strings
- US-032: Implementar manejo de errores robusto y específico

2.2.15. Feature 15

Identificador único (ID): FT-015

Título / Nombre: Escalabilidad y Rendimiento

Descripción: Implementación de estrategias de caché con invalidación basada en eventos, despliegue y escalabilidad mediante Podman.

Épica: EP-002 - Optimización de la Operación y Calidad del Sistema

Prioridad: Media

Historias de Usuario (HUs):

- US-033: Implementar estrategia de caché
- US-034: Desplegar y escalar servicios con Podman

2.2.16. Feature 16

Identificador único (ID): FT-016

Título / Nombre: Confiabilidad y Operación

Descripción: Establecimiento de pipelines de CI/CD, observabilidad completa (logs, métricas, trazabilidad, alertas), health checks y manejo de DLQ. Incluye la resolución del desfase horario en pedidos programados.

Épica: EP-002 - Optimización de la Operación y Calidad del Sistema

Prioridad: Alta

Historias de Usuario (HUs):

- US-035: Establecer pipelines de CI/CD con GitHub Actions
- US-036: Implementar observabilidad completa (logs, métricas, tracing, alertas)
- US-037: Implementar health checks para servicios
- US-038: Manejar mensajes no procesados en RabbitMQ (DLQ)
- US-039: Resolver desfase horario en pedidos programados

2.2.17. Feature 17

Identificador único (ID): FT-017

Título / Nombre: Pruebas Automatizadas

Descripción: Cobertura de pruebas unitarias, de integración y E2E, incluyendo el flujo crítico de pedidos y el worker, e integración de SAST.

Épica: EP-002 - Optimización de la Operación y Calidad del Sistema

Prioridad: Alta

Historias de Usuario (HUs):

- US-040: Cobertura de pruebas unitarias, de integración y E2E
- US-041: Integrar herramientas de análisis estático de seguridad (SAST)

2.2.18. Feature 18

Identificador único (ID): FT-018

Título / Nombre: Experiencia de Desarrollo

Descripción: Integración de linters automáticos y pre-commit hooks para asegurar la calidad del código antes de la subida a repositorio.

Épica: EP-002 - Optimización de la Operación y Calidad del Sistema

Prioridad: Media

Historias de Usuario (HUs):

- US-042: Integrar linters automáticos y pre-commit hooks

2.3. Historias de usuario

2.3.1. Historia de usuario 1

Identificador único (ID): US-001

Título / nombre: Registrarse como cliente

Descripción:

Como **cliente nuevo**,

Quiero **registrarme en la plataforma**,

Para **poder realizar pedidos y gestionar mi cuenta**.

Criterios de aceptación:

- Dado que no tengo una cuenta, cuando proporciono un email y una contraseña válidos (mínimo 12 caracteres, incluyendo al menos una mayúscula, un número y un carácter especial), entonces mi cuenta es creada y puedo iniciar sesión.
- Dado que intento registrarme con un email ya existente, cuando envío el formulario, entonces el sistema me informa que el email ya está en uso.

Prioridad / Orden en el Backlog: Alta

Feature: FT-001 - Gestión de Cuentas de Usuario

Épica: EP-001 - Plataforma de Pedidos en Línea Robusta y Segura

Dependencias: N/A

Versión / Release: 1.0

2.3.2. Historia de usuario 2

Identificador único (ID): US-002

Título / Nombre: Iniciar sesión como cliente/administrador

Descripción:

Como **usuario (cliente o administrador)**,

Quiero **iniciar sesión en la plataforma**,

Para **acceder a las funcionalidades correspondientes a mi rol**.

Criterios de Aceptación:

- Dado que tengo credenciales válidas, cuando las ingreso, entonces accedo al sistema y se me redirige a mi panel.
- Dado que ingreso credenciales inválidas, cuando intento iniciar sesión, entonces el sistema me muestra un mensaje de error.
- Dado que soy un administrador, cuando inicio sesión, entonces accedo al panel administrativo.

Prioridad / Orden en el Backlog: Alta

Feature: FT-001 - Gestión de Cuentas de Usuario

Épica: EP-001 - Plataforma de Pedidos en Línea Robusta y Segura

Dependencias: US-001

Versión / Release: 1.0

2.3.3. Historia de usuario 3

Identificador único (ID): US-003

Título / Nombre: Visualizar el menú de productos

Descripción:

Como **cliente**,

Quiero **visualizar el menú de productos disponibles**,

Para **poder elegir qué pedir**.

Criterios de Aceptación:

- Dado que estoy en la página principal, cuando navego al menú, entonces veo una lista de productos con sus nombres, descripciones y precios.
- Dado que el menú tiene muchos productos, cuando desplazo la página, entonces los productos se cargan de forma eficiente.

Prioridad / Orden en el Backlog: Alta**Feature:** FT-002 - Interfaz de Usuario Cliente**Épica:** EP-001 - Plataforma de Pedidos en Línea Robusta y Segura**Dependencias:** FT-010 (Backend de productos)**Versión / Release:** 1.0**2.3.4. Historia de usuario 4****Identificador único (ID):** US-004**Título / Nombre:** Agregar productos al carrito**Descripción:**Como **cliente**,Quiero **agregar productos a mi carrito de compras**,Para **preparar mi pedido**.**Criterios de Aceptación:**

- Dado que estoy viendo un producto, cuando hago clic en "Agregar al carrito", entonces el producto se añade a mi carrito y se actualiza el total.
- Dado que un producto ya está en el carrito, cuando lo agrego de nuevo, entonces la cantidad del producto en el carrito se incrementa.

Prioridad / Orden en el Backlog: Alta

Feature: FT-002 - Interfaz de Usuario Cliente

Épica: EP-001 - Plataforma de Pedidos en Línea Robusta y Segura

Dependencias: US-003

Versión / Release: 1.0

2.3.5. Historia de usuario 5

Identificador único (ID): US-005

Título / Nombre: Gestionar productos favoritos

Descripción:

Como **cliente**,

Quiero **marcar productos como favoritos, visualizar mi lista de favoritos y eliminar productos de ella**,

Para **acceder rápidamente a ellos en futuras compras**.

Criterios de Aceptación:

- Dado que estoy viendo un producto, cuando hago clic en el icono de "favorito", entonces el producto se añade a mi lista de favoritos.
- Dado que estoy en mi lista de favoritos, cuando hago clic en el icono de "favorito" de un producto, entonces el producto se elimina de mi lista.
- Dado que accedo a la sección de favoritos, cuando la abro, entonces veo todos los productos que he marcado como favoritos.

Prioridad / Orden en el Backlog: Media

Feature: FT-002 - Interfaz de Usuario Cliente

Épica: EP-001 - Plataforma de Pedidos en Línea Robusta y Segura

Dependencias: US-003

Versión / Release: 1.1

2.3.6. Historia de usuario 6

Identificador único (ID): US-006

Título / Nombre: Crear un pedido con dirección de entrega

Descripción:

Como **cliente**,

Quiero **crear un pedido seleccionando una dirección de entrega**,

Para **recibir mi comida en el lugar deseado**.

Criterios de Aceptación:

- Dado que tengo productos en el carrito, cuando procedo al checkout y selecciono una dirección existente, entonces el pedido se registra con esa dirección.
- Dado que no tengo direcciones guardadas, cuando procedo al checkout, entonces puedo añadir una nueva dirección para el pedido.

Prioridad / Orden en el Backlog: Alta

Feature: FT-003 - Gestión de Pedidos (Cliente)

Épica: EP-001 - Plataforma de Pedidos en Línea Robusta y Segura

Dependencias: US-004

Versión / Release: 1.0

2.3.7. Historia de usuario 7

Identificador único (ID): US-007

Título / Nombre: Elegir tipo de pedido (directo o programado)

Descripción:

Como **cliente**,

Quiero **elegir si mi pedido es para entrega "directa" o "programada" (para más tarde)**,

Para **adaptar la entrega a mis necesidades de tiempo**.

Criterios de Aceptación:

- Dado que estoy en el checkout, cuando selecciono "pedido directo", entonces el pedido se prepara inmediatamente.
- Dado que estoy en el checkout, cuando selecciono "programar pedido" y elijo una fecha/hora con un mínimo de 30 minutos de antelación y dentro de las 48 horas siguientes, entonces el pedido se guarda como programado.
- Dado que intento programar un pedido fuera del rango de 30 minutos a 48 horas, cuando selecciono la fecha/hora, entonces la interfaz de usuario deshabilita las opciones fuera de este rango.

Prioridad / Orden en el Backlog: Alta

Feature: FT-003 - Gestión de Pedidos (Cliente)

Épica: EP-001 - Plataforma de Pedidos en Línea Robusta y Segura

Dependencias: US-006, FT-012

Versión / Release: 1.0

2.3.8. Historia de usuario 8

Identificador único (ID): US-008

Título / Nombre: Seleccionar medio de pago

Descripción:

Como **cliente**,

Quiero **seleccionar el medio de pago (ej. efectivo)**,

Para **finalizar mi compra**.

Criterios de Aceptación:

- Dado que estoy en el checkout, cuando selecciono "efectivo" como método de pago, entonces el pedido se confirma con esa opción.
- Dado que el sistema solo soporta efectivo para la V1.0, cuando accedo a la selección de pago, entonces solo veo la opción de "efectivo", con planes de expansión en V1.1+.

Prioridad / Orden en el Backlog: Alta

Feature: FT-003 - Gestión de Pedidos (Cliente)

Épica: EP-001 - Plataforma de Pedidos en Línea Robusta y Segura

Dependencias: US-006

Versión / Release: 1.0

2.3.9. Historia de usuario 9

Identificador único (ID): US-009

Título / Nombre: Aplicar cupones de descuento

Descripción:

Como **cliente**,

Quiero **aplicar cupones de descuento al carrito de compras**,

Para **obtener un precio reducido en mi pedido**.

Criterios de Aceptación:

- Dado que tengo un cupón válido, cuando lo ingreso en el campo correspondiente, entonces el descuento se aplica al total del carrito.
- Dado que ingreso un cupón inválido o expirado, cuando intento aplicarlo, entonces el sistema me informa que el cupón no es válido.
- Dado que el cupón es de porcentaje, cuando se aplica, entonces el descuento se calcula correctamente sobre el total.

Prioridad / Orden en el Backlog: Media

Feature: FT-003 - Gestión de Pedidos (Cliente)

Épica: EP-001 - Plataforma de Pedidos en Línea Robusta y Segura

Dependencias: FT-007

Versión / Release: 1.1

2.3.10. Historia de usuario 10

Identificador único (ID): US-010

Título / Nombre: Consultar el estado de mis pedidos

Descripción:

Como **cliente**,

Quiero **consultar el estado de mis pedidos** (pendiente, en preparación, listo, entregado, programado),

Para **saber cuándo recibiré mi comida**.

Criterios de Aceptación:

- Dado que tengo pedidos activos, cuando accedo a la sección "Mis Pedidos", entonces veo el estado actual de cada uno.
- Dado que el estado de un pedido cambia, cuando consulto la sección "Mis Pedidos" (polling cada 10s), entonces el estado se actualiza en la interfaz.
- Dado que un pedido está programado, cuando consulto su estado antes de la fecha/hora, entonces veo el estado "programado".

Prioridad / Orden en el Backlog: Alta

Feature: FT-003 - Gestión de Pedidos (Cliente)

Épica: EP-001 - Plataforma de Pedidos en Línea Robusta y Segura

Dependencias: FT-012

Versión / Release: 1.0

2.3.11. Historia de usuario 11

Identificador único (ID): US-011

Título / Nombre: Calificar un producto

Descripción:

Como **cliente**,

Quiero **calificar un producto una vez que el pedido ha sido entregado**,

Para **compartir mi opinión y ayudar a otros usuarios**.

Criterios de Aceptación:

- Dado que un pedido ha sido entregado, cuando accedo a los detalles del pedido, entonces puedo dejar una reseña y una calificación para cada producto.
- Dado que ya he calificado un producto, cuando intento calificarlo de nuevo, entonces puedo editar mi reseña existente.

Prioridad / Orden en el Backlog: Media

Feature: FT-003 - Gestión de Pedidos (Cliente)

Épica: EP-001 - Plataforma de Pedidos en Línea Robusta y Segura

Dependencias: FT-008

Versión / Release: 1.1

2.3.12. Historia de usuario 12

Identificador único (ID): US-012

Título / Nombre: Crear, editar y eliminar productos

Descripción:

Como **administrador**,

Quiero **poder crear, editar y eliminar productos**,

Para **mantener el menú actualizado**.

Criterios de Aceptación:

- Dado que estoy en el panel de administración, cuando uso la opción "Crear Producto", entonces puedo añadir un nuevo producto con nombre, descripción, precio e imagen.
- Dado que selecciono un producto existente, cuando uso la opción "Editar Producto", entonces puedo modificar sus detalles.
- Dado que selecciono un producto, cuando uso la opción "Eliminar Producto", entonces el producto se elimina del menú.

Prioridad / Orden en el Backlog: Alta

Feature: FT-005 - Gestión de Productos (Administrador)

Épica: EP-001 - Plataforma de Pedidos en Línea Robusta y Segura

Dependencias: FT-010

Versión / Release: 1.0

2.3.13. Historia de usuario 13

Identificador único (ID): US-013

Título / Nombre: Cargar productos iniciales (semilla)

Descripción:

Como **administrador**,

Quiero **poder cargar productos iniciales mediante un proceso de "semilla"**,

Para **poblar el menú rápidamente al inicio**.

Criterios de Aceptación:

- Dado que el sistema está vacío, cuando ejecuto el script de semilla, entonces la base de datos se llena con un conjunto de productos predefinidos.
- Dado que el script de semilla se ejecuta, cuando consulto el menú, entonces los productos iniciales están visibles.

Prioridad / Orden en el Backlog: Media

Feature: FT-005 - Gestión de Productos (Administrador)

Épica: EP-001 - Plataforma de Pedidos en Línea Robusta y Segura

Dependencias: FT-011

Versión / Release: 1.0

2.3.14. Historia de usuario 14

Identificador único (ID): US-014

Título / Nombre: Visualizar todos los pedidos

Descripción:

Como **administrador**,

Quiero **poder visualizar todos los pedidos**,

Para **supervisar la operación del restaurante**.

Criterios de Aceptación:

- Dado que estoy en el panel de administración, cuando accedo a la sección de pedidos, entonces veo una lista completa de todos los pedidos con sus detalles (cliente, productos, estado, fecha).
- Dado que hay muchos pedidos, cuando uso la paginación o filtros, entonces puedo navegar y buscar pedidos eficientemente.

Prioridad / Orden en el Backlog: Alta

Feature: FT-006 - Gestión de Pedidos (Administrador)

Épica: EP-001 - Plataforma de Pedidos en Línea Robusta y Segura

Dependencias: FT-010

Versión / Release: 1.0

2.3.15. Historia de usuario 15

Identificador único (ID): US-015

Título / Nombre: Cambiar el estado de los pedidos

Descripción:

Como **administrador**,

Quiero **poder cambiar el estado de los pedidos** (pendiente, en preparación, listo, entregado, programado),

Para **gestionar el flujo de trabajo de la cocina y entrega**.

Criterios de Aceptación:

- Dado que estoy viendo un pedido, cuando selecciono un nuevo estado (ej. "en preparación"), entonces el estado del pedido se actualiza en el sistema.
- Dado que un pedido está programado y su fecha/hora se ha cumplido, cuando accedo a él, entonces los botones para cambiar su estado se activan.
- Dado que un pedido está programado y su fecha/hora NO se ha cumplido, cuando accedo a él, entonces los botones para cambiar su estado están deshabilitados (excepto cancelar).

Prioridad / Orden en el Backlog: Alta

Feature: FT-006 - Gestión de Pedidos (Administrador)

Épica: EP-001 - Plataforma de Pedidos en Línea Robusta y Segura

Dependencias: FT-010, FT-012

Versión / Release: 1.0

2.3.16. Historia de usuario 16

Identificador único (ID): US-016

Título / Nombre: Cancelar pedidos programados

Descripción:

Como **administrador**,

Quiero **poder cancelar pedidos programados antes de su fecha/hora de ejecución**,

Para **manejar cambios o imprevistos**.

Criterios de Aceptación:

- Dado que un pedido está programado y su fecha no ha llegado, cuando hago clic en "Cancelar", entonces el pedido se marca como cancelado y no se procesa.
- Dado que un pedido ya ha pasado su fecha/hora programada, cuando intento cancelarlo, entonces la opción de cancelación está deshabilitada.

Prioridad / Orden en el Backlog: Media

Feature: FT-006 - Gestión de Pedidos (Administrador)

Épica: EP-001 - Plataforma de Pedidos en Línea Robusta y Segura

Dependencias: FT-010

Versión / Release: 1.0

2.3.17. Historia de usuario 17

Identificador único (ID): US-017

Título / Nombre: Crear y actualizar cupones de descuento

Descripción:

Como **administrador**,

Quiero **poder crear y actualizar cupones de descuento**,

Para **gestionar promociones y ofertas**.

Criterios de Aceptación:

- Dado que estoy en el panel de administración, cuando uso la opción "Crear Cupón", entonces puedo definir un nuevo cupón con código, tipo (porcentaje/monto fijo), valor y fecha de validez.
- Dado que selecciono un cupón existente, cuando uso la opción "Editar Cupón", entonces puedo modificar sus detalles.

- Dado que un cupón ha expirado, cuando un cliente intenta usarlo, entonces el sistema lo rechaza.

Prioridad / Orden en el Backlog: Media

Feature: FT-007 - Gestión de Cupones (Administrador)

Épica: EP-001 - Plataforma de Pedidos en Línea Robusta y Segura

Dependencias: FT-010

Versión / Release: 1.1

2.3.18. Historia de usuario 18

Identificador único (ID): US-018

Título / Nombre: Listar y eliminar reseñas de productos

Descripción:

Como **administrador**,

Quiero **poder listar y eliminar reseñas de productos**,

Para **moderar el contenido y mantener la calidad**.

Criterios de Aceptación:

- Dado que estoy en el panel de administración, cuando accedo a la sección de reseñas, entonces veo todas las reseñas de productos con su contenido, calificación y autor.
- Dado que una reseña es inapropiada, cuando hago clic en "Eliminar", entonces la reseña se borra del sistema y ya no es visible para los clientes.

Prioridad / Orden en el Backlog: Media

Feature: FT-008 - Gestión de Reseñas (Administrador)

Épica: EP-001 - Plataforma de Pedidos en Línea Robusta y Segura

Dependencias: FT-010

Versión / Release: 1.1

2.3.19. Historia de usuario 19

Identificador único (ID): US-019

Título / Nombre: Visualizar historial de pedidos de clientes

Descripción:

Como **administrador**,

Quiero **poder visualizar el historial de pedidos de los clientes**,

Para **entender sus patrones de compra y ofrecer un mejor servicio**.

Criterios de Aceptación:

- Dado que estoy en el panel de administración, cuando selecciono un cliente, entonces veo una lista de todos sus pedidos anteriores con sus detalles.
- Dado que un cliente tiene muchos pedidos, cuando consulto su historial, entonces puedo filtrar o paginar los resultados.

Prioridad / Orden en el Backlog: Baja

Feature: FT-009 - Gestión de Clientes (Administrador)

Épica: EP-001 - Plataforma de Pedidos en Línea Robusta y Segura

Dependencias: FT-010

Versión / Release: 1.2

2.3.20. Historia de usuario 20

Identificador único (ID): US-020

Título / Nombre: Consolidar lógica de backend duplicada

Descripción:

Como **desarrollador**,

Quiero **eliminar la duplicación de lógica entre backends (api/ y backend/)**,

Para **tener un único backend principal en FastAPI y mejorar la mantenibilidad**.

Criterios de Aceptación:

- Dado que existe lógica duplicada en `api/` y `backend/`, cuando se migra al backend FastAPI, entonces el backend de Express es archivado y la funcionalidad se mantiene.
- Dado que se ha consolidado la lógica, cuando se ejecutan las pruebas, entonces todas las funcionalidades migradas operan correctamente en el backend FastAPI.

Prioridad / Orden en el Backlog: Alta

Feature: FT-010 - Backend / API Principal

Épica: EP-001 - Plataforma de Pedidos en Línea Robusta y Segura

Dependencias: N/A

Versión / Release: 1.0

2.3.21. Historia de usuario 21

Identificador único (ID): US-021

Título / Nombre: Mantener esquemas de datos consistentes

Descripción:

Como **desarrollador**,

Quiero asegurar que los esquemas de datos sean consistentes entre los componentes (Prisma sincronizado),

Para evitar errores de integración y garantizar la integridad de los datos.

Criterios de Aceptación:

- Dado que se realizan cambios en el esquema de la base de datos, cuando se sincroniza Prisma, entonces el esquema de la base de datos es consistente en todos los servicios (backend y worker).
- Dado que los esquemas están sincronizados, cuando los servicios interactúan con la base de datos, entonces no hay errores de mapeo de datos.

Prioridad / Orden en el Backlog: Alta

Feature: FT-011 - Base de Datos

Épica: EP-001 - Plataforma de Pedidos en Línea Robusta y Segura

Dependencias: FT-010, FT-012

Versión / Release: 1.0

2.3.22. Historia de usuario 22

Identificador único (ID): US-022

Título / Nombre: Procesar eventos de pedidos asíncronamente

Descripción:

Como **sistema**,

Quiero **procesar eventos de pedidos de forma asíncrona usando RabbitMQ y un worker**,

Para **desacoplar la creación del pedido de su procesamiento y mejorar el rendimiento**.

Criterios de Aceptación:

- Dado que se crea un pedido en el backend, cuando el backend publica un evento en RabbitMQ, entonces el worker lo consume exitosamente.
- Dado que el worker consume un evento de pedido, cuando lo procesa, entonces el estado detallado del pedido se actualiza correctamente en la base de datos.
- Dado que el procesamiento es asíncrono, cuando se crea un pedido, entonces la respuesta al cliente es rápida sin esperar el procesamiento completo.

Prioridad / Orden en el Backlog: Alta

Feature: FT-012 - Procesamiento Asíncrono de Pedidos

Épica: EP-001 - Plataforma de Pedidos en Línea Robusta y Segura

Dependencias: FT-010, FT-011

Versión / Release: 1.0

2.3.23. Historia de usuario 23

Identificador único (ID): US-023

Título / Nombre: Externalizar y rotar secretos

Descripción:

Como **administrador de seguridad**,

Quiero **proteger los secretos (contraseñas, JWT) externalizándolos y rotándolos**,

Para **evitar su almacenamiento hardcodeado y mejorar la seguridad**.

Criterios de Aceptación:

- Dado que hay secretos en el sistema, cuando se despliega, entonces los secretos se cargan desde variables de entorno o un gestor de secretos, no desde el código o archivos versionados.

- Dado que se necesita rotar un secreto, cuando se actualiza en el gestor, entonces el sistema lo utiliza sin necesidad de redescarga manual.

Prioridad / Orden en el Backlog: Alta

Feature: FT-013 - Seguridad del Sistema

Épica: EP-002 - Optimización de la Operación y Calidad del Sistema

Dependencias: N/A

Versión / Release: 1.0

2.3.24. Historia de usuario 24

Identificador único (ID): US-024

Título / Nombre: Utilizar tokens JWT seguros

Descripción:

Como **administrador de seguridad**,

Quiero **utilizar tokens JWT seguros y con secretos robustos**,

Para **proteger la autenticación de usuarios**.

Criterios de Aceptación:

- Dado que un usuario inicia sesión, cuando se genera un JWT, entonces este utiliza un secreto robusto (no predecible) y tiene una vida útil adecuada (no excesivamente larga).
- Dado que un JWT es inválido o expirado, cuando se presenta al sistema, entonces la autenticación falla.
- Dado que un usuario inicia sesión, cuando se genera un Access Token, entonces este tiene una duración de 15 minutos.
- Dado que un usuario inicia sesión, cuando se genera un Refresh Token, entonces este tiene una duración de 7 días.

Prioridad / Orden en el Backlog: Alta

Feature: FT-013 - Seguridad del Sistema

Épica: EP-002 - Optimización de la Operación y Calidad del Sistema

Dependencias: US-023

Versión / Release: 1.0

2.3.25. Historia de usuario 25

Identificador único (ID): US-025

Título / Nombre: Proteger tokens de sesión contra XSS/CSRF

Descripción:

Como **administrador de seguridad**,

Quiero **proteger los tokens de sesión de vulnerabilidades XSS y CSRF**,

Para **evitar ataques de robo de sesión**.

Criterios de Aceptación:

- Dado que un usuario inicia sesión, cuando se emite un token, entonces este no se almacena en `localStorage` del cliente.
- Dado que se utiliza un token de sesión, cuando se envía al cliente, entonces se usan mecanismos como `HttpOnly` y `SameSite` en las cookies para protegerlo.

Prioridad / Orden en el Backlog: Alta

Feature: FT-013 - Seguridad del Sistema

Épica: EP-002 - Optimización de la Operación y Calidad del Sistema

Dependencias: N/A

Versión / Release: 1.0

2.3.26. Historia de usuario 26

Identificador único (ID): US-026

Título / Nombre: Implementar rotación de credenciales

Descripción:

Como **administrador de seguridad**,

Quiero **implementar la rotación de credenciales**,

Para **reducir el riesgo de compromiso a largo plazo**.

Criterios de Aceptación:

- Dado que las credenciales tienen una vida útil, cuando expiran, entonces el sistema permite su rotación sin interrupción del servicio.
- Dado que se rotan las credenciales, cuando los servicios intentan autenticarse, entonces utilizan las nuevas credenciales correctamente.

Prioridad / Orden en el Backlog: Media

Feature: FT-013 - Seguridad del Sistema

Épica: EP-002 - Optimización de la Operación y Calidad del Sistema

Dependencias: US-023

Versión / Release: 1.1

2.3.27. Historia de usuario 27

Identificador único (ID): US-027

Título / Nombre: Eliminar duplicación de lógica entre backends

Descripción:

Como **desarrollador**,

Quiero **eliminar la duplicación de lógica entre backends (consolidar `api/` y `backend/`)**,

Para **mejorar la mantenibilidad y reducir errores**.

Criterios de Aceptación:

- Dado que existen dos backends con lógica de negocio, cuando se completa la refactorización, entonces solo el backend de FastAPI contiene la lógica de negocio activa y el backend de Express es eliminado o archivado.
- Dado que la lógica ha sido consolidada, cuando se ejecutan las pruebas, entonces todas las funcionalidades migradas operan correctamente en el backend FastAPI.

Prioridad / Orden en el Backlog: Alta

Feature: FT-014 - Mantenibilidad y Calidad de Código

Épica: EP-002 - Optimización de la Operación y Calidad del Sistema

Dependencias: US-020

Versión / Release: 1.0

2.3.28. Historia de usuario 28

Identificador único (ID): US-028

Título / Nombre: Adherirse al principio de responsabilidad única

Descripción:

Como **desarrollador**,

Quiero **asegurar que las funciones se adhieran al principio de responsabilidad única**,

Para **mejorar la claridad y facilitar el mantenimiento del código**.

Criterios de Aceptación:

- Dado que se revisa el código, cuando se evalúan las funciones, entonces cada función tiene una única razón para cambiar y realiza una tarea específica.
- Dado que se implementan nuevas funcionalidades, cuando se desarrollan, entonces se diseñan siguiendo el principio de responsabilidad única.

Prioridad / Orden en el Backlog: Media**Feature:** FT-014 - Mantenibilidad y Calidad de Código**Épica:** EP-002 - Optimización de la Operación y Calidad del Sistema**Dependencias:** N/A**Versión / Release:** 1.1**2.3.29. Historia de usuario 29****Identificador único (ID):** US-029**Título / Nombre:** Usar logging estructurado en producción**Descripción:**Como **desarrollador**,

Quiero **usar logging estructurado en producción en lugar de** `print()` o `console.log()`,

Para **facilitar el análisis y la depuración de problemas**.

Criterios de Aceptación:

- Dado que el sistema está en producción, cuando se generan eventos, entonces los logs se emiten en un formato estructurado (ej. JSON) y no como texto plano.

- Dado que se revisa el código, cuando se encuentran `print()` o `console.log()` para logging, entonces se reemplazan por una solución de logging estructurado.

Prioridad / Orden en el Backlog: Alta

Feature: FT-014 - Mantenibilidad y Calidad de Código

Épica: EP-002 - Optimización de la Operación y Calidad del Sistema

Dependencias: FT-016

Versión / Release: 1.0

2.3.30. Historia de usuario 30

Identificador único (ID): US-030

Título / Nombre: Evitar el uso indiscriminado de `any` en TypeScript

Descripción:

Como **desarrollador**,

Quiero **evitar el uso indiscriminado de `any`** en el código TypeScript,

Para **mejorar la seguridad de tipos y la calidad del código**.

Criterios de Aceptación:

- Dado que se revisa el código TypeScript, cuando se encuentran usos de `any`, entonces se refactorizan para usar tipos específicos o genéricos.
- Dado que se desarrolla nuevo código TypeScript, cuando se escribe, entonces se evita el uso de `any` a menos que sea estrictamente necesario y justificado.

Prioridad / Orden en el Backlog: Media

Feature: FT-014 - Mantenibilidad y Calidad de Código

Épica: EP-002 - Optimización de la Operación y Calidad del Sistema

Dependencias: N/A

Versión / Release: 1.1

2.3.31. Historia de usuario 31

Identificador único (ID): US-031

Título / Nombre: Evitar SQL embebido en strings

Descripción:

Como **desarrollador**,

Quiero **evitar SQL embebido en strings**,

Para **prevenir inyecciones SQL y mejorar la seguridad**.

Criterios de Aceptación:

- Dado que se interactúa con la base de datos, cuando se construyen consultas, entonces se utilizan ORMs (como Prisma) o consultas parametrizadas.
- Dado que se revisa el código, cuando se encuentran strings SQL embebidos, entonces se refactorizan para usar métodos seguros de acceso a datos.

Prioridad / Orden en el Backlog: Alta

Feature: FT-014 - Mantenibilidad y Calidad de Código

Épica: EP-002 - Optimización de la Operación y Calidad del Sistema

Dependencias: N/A

Versión / Release: 1.0

2.3.32. Historia de usuario 32

Identificador único (ID): US-032

Título / Nombre: Implementar manejo de errores robusto y específico

Descripción:

Como **desarrollador**,

Quiero **implementar un manejo de errores robusto y específico**,

Para **proporcionar mensajes claros y facilitar la depuración**.

Criterios de Aceptación:

- Dado que ocurre un error en el sistema, cuando el sistema lo maneja, entonces se registra un error específico con contexto y se devuelve una respuesta adecuada al usuario (sin exponer detalles internos).
- Dado que se desarrolla nuevo código, cuando se implementa, entonces incluye bloques de manejo de errores específicos para diferentes escenarios.

Prioridad / Orden en el Backlog: Alta

Feature: FT-014 - Mantenibilidad y Calidad de Código

Épica: EP-002 - Optimización de la Operación y Calidad del Sistema

Dependencias: N/A

Versión / Release: 1.0

2.3.33. Historia de usuario 33

Identificador único (ID): US-033

Título / Nombre: Implementar estrategia de caché

Descripción:

Como **desarrollador**,

Quiero **implementar una estrategia de caché**,

Para **mejorar el rendimiento del sistema y reducir la carga de la base de datos**.

Criterios de Aceptación:

- Dado que se accede a datos frecuentemente (ej. menú de productos), cuando se implementa caché, entonces las solicitudes repetidas se sirven más rápido desde la caché.
- Dado que los datos en caché se actualizan en la base de datos, cuando se invalida la caché, entonces se utiliza un mecanismo de invalidación basado en eventos (ej. publicación de eventos de actualización) para servir los datos más recientes.

Prioridad / Orden en el Backlog: Media

Feature: FT-015 - Escalabilidad y Rendimiento

Épica: EP-002 - Optimización de la Operación y Calidad del Sistema

Dependencias: N/A

Versión / Release: 1.1

2.3.34. Historia de usuario 34

Identificador único (ID): US-034

Título / Nombre: Desplegar y escalar servicios con Podman

Descripción:

Como **operador**,

Quiero **poder desplegar y escalar el sistema mediante Podman**,

Para **garantizar la flexibilidad y disponibilidad de los servicios**.

Criterios de Aceptación:

- Dado que se necesita desplegar el sistema, cuando se ejecuta `podman-compose.yml`, entonces todos los servicios se inician correctamente.
- Dado que se necesita escalar un servicio, cuando se ajusta la configuración de Podman, entonces el servicio se escala horizontalmente sin interrupciones.

Prioridad / Orden en el Backlog: Alta

Feature: FT-015 - Escalabilidad y Rendimiento

Épica: EP-002 - Optimización de la Operación y Calidad del Sistema

Dependencias: N/A

Versión / Release: 1.0

2.3.35. Historia de usuario 35

Identificador único (ID): US-035

Título / Nombre: Establecer pipelines de CI/CD con GitHub Actions

Descripción:

Como **desarrollador/DevOps**,

Quiero **establecer pipelines de Integración Continua (CI) y Despliegue Continuo (CD) con GitHub Actions**,

Para **automatizar la validación y los despliegues**.

Criterios de Aceptación:

- Dado que se realiza un commit a la rama principal, cuando se ejecuta el pipeline de CI, entonces las pruebas unitarias, de integración y de seguridad se ejecutan automáticamente.
- Dado que el pipeline de CI pasa, cuando se cumplen las condiciones de despliegue, entonces el pipeline de CD despliega el código automáticamente.

Prioridad / Orden en el Backlog: Alta

Feature: FT-016 - Confiabilidad y Operación

Épica: EP-002 - Optimización de la Operación y Calidad del Sistema

Dependencias: US-040, US-041, US-042

Versión / Release: 1.0

2.3.36. Historia de usuario 36

Identificador único (ID): US-036

Título / Nombre: Implementar observabilidad completa (logs, métricas, tracing, alertas)

Descripción:

Como **operador**,

Quiero **implementar observabilidad completa (logs estructurados, métricas, trazabilidad distribuida, alertas)**,

Para **monitorear el estado y rendimiento del sistema en producción.**

Criterios de Aceptación:

- Dado que el sistema está en producción, cuando ocurren eventos, entonces se generan logs estructurados que pueden ser consultados en una herramienta de agregación.
- Dado que los servicios están en ejecución, cuando se monitorean, entonces se recogen métricas de rendimiento y uso de recursos.
- Dado que se realiza una solicitud, cuando se procesa a través de múltiples servicios, entonces se puede seguir su trazabilidad distribuida.
- Dado que una métrica excede un umbral, cuando se configura una alerta, entonces se notifica al equipo de operaciones.

Prioridad / Orden en el Backlog: Alta

Feature: FT-016 - Confiabilidad y Operación

Épica: EP-002 - Optimización de la Operación y Calidad del Sistema

Dependencias: US-029

Versión / Release: 1.0

2.3.37. Historia de usuario 37

Identificador único (ID): US-037

Título / Nombre: Implementar health checks para servicios

Descripción:

Como **operador**,

Quiero **implementar health checks para todos los servicios**,

Para **monitorear su disponibilidad y estado de forma proactiva**.

Criterios de Aceptación:

- Dado que un servicio está en ejecución, cuando se consulta su endpoint de health check, entonces devuelve un estado que indica su correcto funcionamiento.
- Dado que un servicio tiene un problema, cuando se consulta su health check, entonces devuelve un estado de error.

Prioridad / Orden en el Backlog: Alta

Feature: FT-016 - Confiabilidad y Operación

Épica: EP-002 - Optimización de la Operación y Calidad del Sistema

Dependencias: N/A

Versión / Release: 1.0

2.3.38. Historia de usuario 38

Identificador único (ID): US-038

Título / Nombre: Manejar mensajes no procesados en RabbitMQ (DLQ)

Descripción:

Como **operador**,

Quiero **manejar mensajes no procesados en RabbitMQ mediante una Dead Letter Queue (DLQ)**,

Para **evitar la pérdida de datos y facilitar la depuración de errores asíncronos**.

Criterios de Aceptación:

- Dado que un mensaje no puede ser procesado por el worker después de varios reintentos, cuando falla, entonces se redirige automáticamente a la DLQ.
- Dado que hay mensajes en la DLQ, cuando se revisa, entonces se puede analizar la causa del fallo y reprocesar si es necesario.

Prioridad / Orden en el Backlog: Media

Feature: FT-016 - Confiabilidad y Operación

Épica: EP-002 - Optimización de la Operación y Calidad del Sistema

Dependencias: FT-012

Versión / Release: 1.1

2.3.39. Historia de usuario 39

Identificador único (ID): US-039

Título / Nombre: Resolver desfase horario en pedidos programados

Descripción:

Como **desarrollador**,

Quiero **resolver el posible problema de zona horaria que causa un desfase de ~3 horas en pedidos programados**,

Para **asegurar la correcta ejecución de los pedidos en el tiempo esperado**.

Criterios de Aceptación:

- Dado que se programa un pedido para una hora específica, cuando se ejecuta, entonces se activa exactamente a la hora programada sin desfases por zona horaria.
- Dado que se prueba la programación de pedidos en diferentes zonas horarias, cuando se ejecutan, entonces la hora de activación es consistente.

Prioridad / Orden en el Backlog: Alta

Feature: FT-016 - Confiabilidad y Operación

Épica: EP-002 - Optimización de la Operación y Calidad del Sistema

Dependencias: FT-003

Versión / Release: 1.0

2.3.40. Historia de usuario 40

Identificador único (ID): US-040

Título / Nombre: Cobertura de pruebas unitarias, de integración y E2E

Descripción:

Como **desarrollador**,

Quiero **tener cobertura de pruebas unitarias, de integración y End-to-End (E2E)**,

Para **garantizar la estabilidad y el correcto funcionamiento del sistema**.

Criterios de Aceptación:

- Dado que se ejecutan las pruebas unitarias, cuando se completan, entonces se alcanza un porcentaje de cobertura definido (ej. >80%) y todas pasan.
- Dado que se ejecutan las pruebas de integración, cuando se completan, entonces la interacción entre componentes funciona correctamente.
- Dado que se ejecutan las pruebas E2E, cuando se completan, entonces el flujo crítico de pedidos (cliente y administrador) funciona de principio a fin.

Prioridad / Orden en el Backlog: Alta

Feature: FT-017 - Pruebas Automatizadas

Épica: EP-002 - Optimización de la Operación y Calidad del Sistema

Dependencias: N/A

Versión / Release: 1.0

2.3.41. Historia de usuario 41

Identificador único (ID): US-041

Título / Nombre: Integrar herramientas de análisis estático de seguridad (SAST)

Descripción:

Como **desarrollador de seguridad**,

Quiero **integrar herramientas de análisis estático de seguridad (SAST)** de forma **automatizada**,

Para **detectar vulnerabilidades en el código fuente tempranamente**.

Criterios de Aceptación:

- Dado que se realiza un commit, cuando se ejecuta el pipeline de CI, entonces la herramienta SAST analiza el código y reporta posibles vulnerabilidades.

- Dado que la herramienta SAST detecta una vulnerabilidad, cuando se genera el informe, entonces se proporciona información detallada para su corrección.

Prioridad / Orden en el Backlog: Alta

Feature: FT-017 - Pruebas Automatizadas

Épica: EP-002 - Optimización de la Operación y Calidad del Sistema

Dependencias: US-035

Versión / Release: 1.0

2.3.42. Historia de usuario 42

Identificador único (ID): US-042

Título / Nombre: Integrar linters automáticos y pre-commit hooks

Descripción:

Como **desarrollador**,

Quiero **integrar linters automáticos y pre-commit hooks**,

Para **asegurar la calidad del código y la detección temprana de errores antes de la subida a repositorio**.

Criterios de Aceptación:

- Dado que se intenta hacer un commit, cuando se ejecuta el pre-commit hook, entonces el código se formatea y se valida según las reglas del linter.
- Dado que el linter detecta un error de estilo o un problema de calidad, cuando se ejecuta, entonces el commit es bloqueado hasta que se corrige.

Prioridad / Orden en el Backlog: Alta

Feature: FT-018 - Experiencia de Desarrollo

Épica: EP-002 - Optimización de la Operación y Calidad del Sistema

Dependencias: N/A

Versión / Release: 1.0

2.4. Matriz de riesgos

2.4.1. Matriz de evaluación de riesgos

ID riesgo	Nombre del riesgo	Probabilidad	Impacto	Nivel de riesgo	Posible mitigación
RS-001	Secretos hardcodeados o accesibles	Media	Alta	Alto	Externalizar y rotar secretos
RS-002	Vulnerabilidades XSS/CSRF por JWT en localStorage	Media	Alta	Alto	No usar localStorage; HttpOnly/SameSite
RS-003	Duplicación de lógica de backend	Alta	Media	Medio	Consolidar backend en FastAPI
RS-004	Falta de automatización CI/CD	Alta	Media	Medio	Implementar GitHub Actions
RS-005	Falta de observabilidad (logs, métricas, tracing)	Alta	Media	Medio	Implementar observabilidad completa
RS-006	Inconsistencia de esquemas de BD	Media	Alta	Alto	Sincronizar Prisma; Validaciones

ID riesgo	Nombre del riesgo	Probabilidad	Impacto	Nivel de riesgo	Posible mitigación
RS-007	Desfase horario en pedidos programados	Media	Media	Medio	Ajustar manejo de zonas horarias
RS-008	Falta de cobertura de pruebas	Alta	Alta	Alto	Implementar pruebas unitarias, integración, E2E
RS-009	Dependencia de recursos humanos con experiencia	Media	Alta	Alto	Capacitación; Contratación; Planificación
RS-010	Mensajes no procesados en RabbitMQ	Media	Media	Medio	Implementar Dead Letter Queue (DLQ)
RS-011	Vulnerabilidades por SQL embebido	Media	Alta	Alto	Usar ORMs o consultas parametrizadas
RS-012	Uso indiscriminado de `any` en TypeScript	Alta	Baja	Bajo	Refactorizar; Linters estrictos

3. Información del Proyecto

Proyecto: SoftDomiFood

Proceso completado: 15 de diciembre de 2025, 19:27

Generado por Agent IRIS by Sofka

Documento generado automáticamente

Proceso completado: 15/12/2025

sofka_